

Ipaper

IPAPER PIPELINE

****Recent Code Changes:****

****26-03-2025 - Modularisation Phase 2:****

- Modularised Global rules, Exceptions, Retailer rules and Store details
- Created separate notebooks for each module with functions to be called: global_rules.py, exceptions.py, retailer_rules.py, Store_details.py
- Store details function is same for all notebooks
- merge_plans_bounty function is used for bounty-specific plan merging

****27-07-2025 - Yearly Plan Changes:****

- Added yearly plan handling for enrollment_plan
- For yearly plans (plan_bill_frequency = 'Yearly' and enrollment_plan_bill_frequency = 'Yearly'), enrollment_plan is divided by 12
- Added new global rule (yearly_plan_non_us) to eliminate Yearly Plan records in countries other than US

****MSH Germany Grossup Logic:****

- Added consent percentage calculation for MSH Germany
- retailer_mix_round_off_grossup is calculated for MSH DE subscribers
- ipaper_amount_grossup for DE = retailer_mix_round_off_grossup * 2

Pipeline data flow structure -

1)Start -

2)Source Data -

Data from team_iide_prod.gold_ii.iink_sub_base_monthly_snapshot and team_iipa_prod.retailer_compensation.cross_country_compensation
Read data from catalog using read_from_catalog function.
Write data to delta tables using upsert function.

****New columns added to source query:****

- enrollment_plan_bill_frequency, plan_bill_frequency (for Yearly plans)

3)Define Parameters –

We will pass the following parameters:

InputDict – Used to mention all the countries and retailers.
StartDateList – Used for custom catch up.If not mentioned default is 3months.
EndDateList – Used for custom catch up.If not mentioned default is last date of the reporting month.
Program_typeList – InstantInk,Instant Toner ,SMB Toner Mono,SMB Toner colour.
Reporting_month – The month for which pipeline runs.
FreezeMix – For yes, last months mix will be taken and no mix will be calculated.For No, new mix will be calculated.
refresh_type – Options: Audit, dev, Standard, testing. Controls checkpoint data saving behavior.
feature_name – Feature name for dev testing.
table_names – Table names for dev testing.
Required libraries should be imported.
Adding delta storage paths and add configuration parameters for better performance.

****Modular Imports:****

The following common utility notebooks are imported:

- common_utils - Common utility functions
- global_rules - Global rules for eligibility (global_rules_columns function)
- exceptions - Exception handling (exceptions_csv, check_and_apply_exceptions functions)
- retailer_rules - Retailer-specific rules (load_retailer_rules, process_retailer_rules, load_retailer_values functions)
- Store_details - Store information lookup (add_store_details, merge_plans_bounty functions)

4)Add Custom Calendar for US - office depot and staples –

Table - team_iipa_prod.app_bm_instant_ink_ops.custom_accounting_calendar

- 1.Getting latest snapshot data from team_iide_prod.gold_ii.iink_sub_base_monthly_snapshot.
- 2.Changing enrolled_on_date for flip and hybrid flip subscribers to their compensation_start_date as this is their full conversion rate to Instant Ink customer.
- 3.Getting country and region from cross country table-retailer_compensation.cross_country_compensation (prod/Dependencies/Cross Country/Cross Country Pipeline and creating new country_id and region_id).
- 4.Use sql query to fetch data for recent snapshot date from "team_iide_prod.gold_ii.iink_sub_base_monthly_snapshot" table. Filter out cross_country_comp_flag = 'Yes' and if flip_type is flip, hybrid flip then compensation_start_date else enrolled_on_date.
- 5.Reading all the input parameters passed.
- 6.Calculate the start and end dates for data processing- If Reporting_month is given then start and end dates are calculated by adding 3 months standard

catch up. If not standard catch up then start and end dates needs to be given as input accordingly.

7.If start_date_list and end_date_list are not provided it calculates default data range i.e 3 months based on reporting month.

8.Modify start date and end date to consider custom calendar from "app_bm_instant_ink_ops.custom_accounting_calendar" - being updated manually based on calendars shared by CO.

Link: <https://hp.sharepoint.com/:f:/r/teams/PAInstantInkTeam/Shared%20Documents/General/Custom%20Calendar?csf=1&web=1&e=LIMd9k>

9.Next processes the data from custom accounting calendar.

What is custom calendar- For US staples, Office Depot, the start_Date_List and end_Date_List changes according to the data given in custom calendar table due to business requirements.

10.QueryCompleterPrivacy- is used to filter entire countries data even if we run the pipeline for a subset of retailers as we need all the retailer's data for privacy phase 2 allocation.

11.QueryCompleter-is used to filter the country as well as the retailers, these we give as the input parameters.

12.Functions are used to handle different scenarios such as matched updates, inserts and updates for records do not present in the source. Tracking updates and inserts with timestamps and version numbers ensuring data consistency and traceability. And also, delta lakes 'merge' functionality helps in handling update and insert logic.

13.Log Creator- used to create a log of all the delta tables written, it adds few additional columns like what filters were used to run the pipeline(country, date, retailer),creation time, previous version number to the existing ones we get in the delta table history.

Upsert -

If PkeyCombo exists, then update.

If PkeyCombo does not exist, then insert.

If data does not exist in current version but exist in previous version, then unmatched (happens because of source change or rules updating).

If data does not exist in current version because we haven't run the current pipeline for a specific retailer or PKeyCombo then irrelevant.

14.Read data from snapshot and remove any duplicate rows.

15.Then few transformations are done for data movement as per requirement.

```
cust_sub_base = cust_sub_base.withColumn('enrollment_plan',regexp_replace('enrollment_plan', ' Page Plan', ''))
```

This line uses the withColumn function to replace the string " Page Plan" with an empty string in the 'enrollment_plan' column of cust_sub_base. The regexp_replace function is used to perform the replacement. For e.g. the column value '100 Page Plan' will be made to '100'

```
cust_sub_base = cust_sub_base.withColumn('reporting_plan',regexp_replace('reporting_plan', ' Page Plan', ''))
```

Similar to the previous step, this line replaces the string " Page Plan" with an empty string in the 'reporting_plan' column of cust_sub_base.

```
cust_sub_base = cust_sub_base.withColumn('reporting_month', lit(reporting_month))
```

This line adds a new column called 'reporting_month' to cust_sub_base and sets its value to the reporting_month variable taken as input parameter.

```
cust_sub_base = cust_sub_base.withColumn('enrolled_on_month',regexp_replace(cust_sub_base["enrolled_on_date"]:[0:7],'-',''))
```

This line adds a new column called 'enrolled_on_month' to cust_sub_base. It extracts the first seven characters from the 'enrolled_on_date' column, represents it as a string, and then replaces any hyphens ('-') with an empty string.

****NEW (27-07-2025) - Yearly Plan Enrollment Plan Adjustment:****

For yearly plans, the enrollment_plan is divided by 12:

- If plan_bill_frequency = 'Yearly' AND enrollment_plan_bill_frequency = 'Yearly', then enrollment_plan = enrollment_plan / 12

- Otherwise, enrollment_plan remains unchanged

```
cust_sub_base = cust_sub_base.withColumn('plan_pages_proxy',when(col('enrollment_plan')==10,15).when(col('enrollment_plan')==25,50)...otherwise(col('enrollment_plan')))
```

This line adds a new column called 'plan_pages_proxy' to cust_sub_base.

It uses when function to create a column that maps specific values of the 'enrollment_plan' column to new values. If 'enrollment_plan' is '10', it is replaced with '15'. If 'enrollment_plan' is '25', it is replaced with '50'. If 'enrollment_plan' is '500', it is replaced with '300'. If 'enrollment_plan' is '1500', it is replaced with '700'. For any other value, the 'enrollment_plan' is kept as is. for phase 2 allocation as 500/1500-page plan had low enrollees hence combining them.

```
-cust_sub_base = cust_sub_base.withColumn("relevant_month",trunc(col('enrolled_on_date'), "month"))
```

Finally, this line adds a new column called 'relevant_month' to cust_sub_base. It uses the trunc function to truncate the 'enrolled_on_date' column to the month level.This means that the day and time components of the date are discarded, and only the year-month portion remains in the new column.

5)Global Rules flag created -

Boolean flags are created for each global rule criteria detailed document on Global rules business definitions.

Link :

<https://hp.sharepoint.com/p:/r/teams/PAInstantInkTeam/Shared%20Documents/General/Code%20Review-Ph2-Feb24/RC%20Code%20and%20Rules%20Review.pptx?d=w7b24fa25f6364364b3115fc3e64139ce&csf=1&web=1&e=69gtCT>

bountry_rules_criteria_combined- created based on printer_replacement, billable_customer_bounty,kit_enrollment,flip_enrollment_all & kitless_21_da.

Global rules : Below mentioned are global rules which needs to be checked.

Printer_replacement:In this case replacement_index is null or replacement_index is

1.One time bounty across lifetime, No bounty at the time of printer replacement.

2)billable_customer_bounty:In this case subscription_type is Billable Customer or subscription_type is HP Employees. Bounty paid to Billable Customer & HP Employee only.

3)kit_enrollment: In this case enrollment_typeis not equal to Kitless and flip_type not equal to Flip and flip_type not equal to Hybrid Flip.

4)flip_enrollment_all:In this flip_type is equal to Flip or flip_type is equal to Hybrid Flip and flip_compensation_eligible_flag is True.

Bounty Paid to all Flip customers converts within trial months + 21 days grace period (logic embedded in IIBI source)

5)kitless_21_days:In this enrollment_type is Kitless and p2_enrollment is like P1% and flip_type not equal to Flip and flip_type not equal to Hybrid Flip.

Bounty paid to all Kit enrollments (both P1 & P2) + P1 Kitless enrollments

6)is_billable_customer :In this subscription_type is equal to Billable Customer.

7)is_HP_employee:In this case subscription_type is equal to HP Employees.

8)is_kitless_enrollment: In this case enrollment_type is equal to Kitless.

9)is_Flip: In this acse flip_type is equal to Flip.

10)is_HybridFlip:In this case flip_type is equal to Hybrid Flip.

11)is_Flip_compensation_eligible:In this case flip_compensation_eligible_flag is equal to True.

12)is_P1:In this case p2_enrollment is like P1%.

13)**yearly_plan_non_us** (NEW - 27-07-2025): When enrollment_plan_bill_frequency is 'Yearly' AND country_id is NOT 'US' - eliminates yearly plan records for non-US countries

14) bountry_rules_criteria_combined:In this case printer_replacement and billable_customer_bounty \ and kit_enrollment or flip_enrollment_all or kitless_21_days AND NOT yearly_plan_non_us.

6) Global_rules_exceptions_combined_is_created –

1.Check eligibility exceptions for bounty from “conditional_exceptions.csv”.Add conditions according to the csv file for retailers Dixons country GB and retailer MSH country DE.

2.Then convert the Data Frame fixed_exceptions_list into a list of dictionaries, where each dictionary represents a row of the Data Frame with column names as keys.

3.If action is none then we get the eligibility exception null and for that retailer if it's not null then conditions are specified where retailer kit printer from source is matched with csv file retailer then we update exception name i.e xmo2_exception , MSH_Exception.

4.Get exceptions from app_bm_instant_ink_ops.exceptions_list. If there are any nulls in exception end date, fill them with the maximum date, i.e tomorrow's date add eligibility_exception in cust_sub_base by joining on exception_value(row with column mapping),printer_retailer_source, exception_start_date<=enrolled on date<=exception_end_date and country id.

exception_field and exception_column should match
printer_retailer_source should match with the source from exceptions table
enrolled_on_date should be between exception_start_date and exception_end_date
country_id should match with the country from exceptions table
If all the above conditions are met, write value of exception_field in eligibility_exception

5.Create global_rules_exceptions_combined in cust_sub_base which is true if bountry_rules_criteria_combined is true and 'eligibility_exception' column is either an empty string (") or null (isNull()). else false.

6)Ipaper source data –

Table - team_iide_prod.gold_ii.rc_sub_base_vw

1.From the table team_iide_prod.gold_ii.rc_sub_base_vw extract ipaper data for further tranformations.

****Ipaper Eligibility Criteria:****

- program_type = 'Instant Ink'

- ipaper_activation_ts or ipaper_join_ts is not null

- latest_enrolled_instant_paper_state = 'activated'
- replacement_index is null or equals 1

7) Transform data –

1. After extracting data from the table bi_fact.rc_sub_base_vw few transformations are done for analysing data as per business requirements.
2. Transformations are present in ipaper code.

8) Create ipaper_bounty_eligibility –

1. Create a column where ipaper county eligibility is checked.
2. If ipaper is given those data are been processed further.

Here after the data is fetched is combined with global_rules_exceptions_combined is created.

9) Ipaper Data –

1. Here after the data is fetched is from Create ipaper_bounty_eligibility step and combined with global_rules_exceptions_combined is created step.
2. Combination of these data is known as Ipaper data.

10) Check point – 0

This end check point 0, all these data come into df0.

Source data added.
Global rules flag added.

11) Retailer Rules columns are added –

Tables -

mapping_rules_index,
mapping_retailer_compensation,
mapping_retailer_compensation_rule

1. Data from source with global rules flag will be present in df0.
2. Add retailer rules using modular functions:
 - load_retailer_rules() - Loads retailer rules with additional columns for ipaper: plan_price as planprice, ipaper_bounty, ipaper_start_date, ipaper_enddate
 - process_retailer_rules() - Processes name value key for rules
 - load_retailer_values() - Loads retailer values table
3. From mapping_rules_index table we get retailer comp index, retailer rules index, rule start date ,rule end date for those only records where status = Approved.
4. We get generic retailer details from this table and only for "ES" country data will be present others will be null, mapping_retailer_compensation will give reporting timeline data i.e monthly, quarterly, half yearly and yearly.
5. mapping_retailer_compensation_rule - will give plan pages, plan price, kit bounty, kitless bounty, kit revshare, kitless revshare, contract start date, contract end date, rules hierarchy.
6. We filter contract_status not equal to no contract for the above table.
7. Retailer rules are being added with few transformations are made as per business requirement.
8. From final retailer rules table, a distinct count of generic retailer details and printer shipment source where they are not equal to unavailable for each bounty rule enrollment plan. This is done for Spain taxation purpose.
9. We make sure that the count in retailer rules and cus_sub_base are same (refer Bounty code) by joining both the tables.
10. Before mix calculation we take global rules exception combined true and we divide the data according to consented and non - consented data. Only the non - consented is used for mix calculation.

****Ipaper Date Eligibility:****

In addition to ruleDateEligibility, ipaperDateEligibility is created by checking if enrolled_on_date falls between ipaper_start_date and ipaper_end_date from the retailer rules.

12) Retailer_mix calculations -

Mix Calculation:

Create country_df by Grouping country_id, relevant_month, plan_pages_proxy, program_type, reporting_month and number of records for each group is counted and stored in a column named country_count.
Create ret_df by grouping country_id, relevant_month, retailer_kit_printer, plan_pages_proxy, program_type, reporting_month,

retailer_kit_printer_org_details_mod, printer_retailer_source_mod and number of records for each group is counted and stored in a column named retailer_count.

If freezeMix is 'No' then :

The ret_df DataFrame is left-joined with the country_df DataFrame on the columns: country_id, relevant_month, plan_pages_proxy, program_type, and reporting_month.

Calculating retailer_mix: A new column retailer_mix is created, which is the ratio of retailer_count to country_count. This ratio represents the proportion of the retailer's count relative to the country's total count. The relevant columns are selected from the joined DataFrame and Some columns are renamed to make them more descriptive.

If freezeMix is 'Yes' then pre-calculated data from the previous month is loaded from a Delta table stored at a specified location (baseDestLoc + 'priv_mix_calculated').

Mix Application:

Allocating Non-Consented Data: This operation allocates non-consented customer data to retailers based on the previously calculated mix ratios.

For the consented data, this operation assigns the allocation directly based on existing retailer data since it's already consented. The retailer_mix is set to 1, meaning the entire allocation goes to the specified retailer without any further ratio calculation.

The combined DataFrame final_eligible_allocated now contains all eligible customer records, both consented and non-consented, along with their respective allocations.

The combined DataFrame is registered as a temporary SQL view named final_eligible_allocated_sql

The initial query string query_filter_relevant_eligible_allocated is a simple SELECT * query designed to select all records from the temporary view.

The final SQL query is executed using spark.sql, returning a DataFrame df_eligible_allocated_relevant_filtered that contains the filtered results.

Define Retailer rules and divide data into attributed and non-attributed post applying for global rules and exceptions. Retailer mix is calculated for unconsented using consented as retailer count/country count and then applied to consented cohort.

13)Check point – 1

This end check point 1, all these data come into df1.

Global rules pass.

Allocation is done.

14)Join retailer rules with result of allocated data -

1.Joining data post allocation with final retailer rules data on country , allocated_retailer,allocated_details,allocated_source,reporting_plan to get data from rules data table, rule_date_eligibility is generated by checking if enrolled on date is between rule start date and rule end date.

15)Create rule_data_eligibility -

1.Joining retailer rule with cust_sub_base tables and creating a column called rule date eligibility.

2.Rule date eligibility is created by checking if enrolled on date starts between rule start date and rule end date.

16) check point - 2

This end check point 2, all these data come into df2.

Retailer rules added.

Rule data eligibility flag added.

17)Rule_Date_Eligibility is True? -

1.Filtering cases only where rule date eligibility AND ipaper date eligibility is true. We filter the most recent rule.

****Both conditions must be True:****

- ruleDateEligibility: enrolled_on_date is between rule_start_date and rule_end_date

- ipaperDateEligibility: enrolled_on_date is between ipaper_start_date and ipaper_end_date

18)Create compensation_type -

1.Now we create Compensation type and categorize data if its kit bounty, kitless bounty or Not Applicable. It is decided by the following conditions:

Kit bounty: When enrollment_type is kit and enrolled_on_date is between contract start date and contract end date.

Kitless bounty: When enrollment_type is kitless and enrolled_on_date is between contract start date and contract end date.

Not Applicable: If it doesn't fall under both of the cases above.

2.We create json rules criteria is created as pass.

19)Create Contract_Eligibility_Criteria -

1.Contract_eligibility_criteria is created as 'Pass' for all compensation_type except 'Not Applicable', for which it is fail.

20)Create Bounty_Eligibility -

1.Here we decide the bounty eligibility if it is given, not given or eligible but no bounty:

Eligible but no bounty: When compensation type is Kit bounty and kit bounty is 0 or when compensation type is kitless bounty and kitless bounty is 0 and

json rules criteria is pass and contract eligibility criteria is pass.
Bounty given: json rules criteria is pass and contract eligibility criteria is also pass.
No Bounty: When both the above conditions are not satisfied.

21)Check point – 3

This end check point 3, all these data come into df3.

Retailer rule date eligibility is true is passed.
Bounty eligibilty flag is created.
Contract eligibility criteria is verified.

22)Get Frequency of reports -

1.Join with mapping_rules_index to get frequency related information.

23)Filter Rules with highest Hierarchy -

1.For country TW we change the rule end date to 2023-11-03 and We filter the rules with maximum hierarchy.

24)Bounty_Eligibility : is Bounty given -

Filtering for only bounty_eligibility i.e 'Bounty given' AND ipaper_bounty_eligibility i.e 'Ipaper Bounty Given'.

****Both conditions must be met for final iPaper bounty:****

- bounty_eligibility = 'Bounty given'
- ipaper_bounty_eligibility = 'Ipaper Bounty Given'

25)Store details added -

1.For store details,get store details from "app_instant_ink_bi_fact.stores_per_subid"

2.We transform these columns oobe_timestamp, cancellation_date into date format.We join the tore table with result on subscription_id.We create 4 columns which are store id ,store key,store display name,store comments text.

3.To create store id these are the conditions:

If the value of the column 'enrollment_type' is 'Prepaid_Standalone', and the 'ori_card_store_id' is null while the 'ori_printer_store_id' is not null, then the value of the 'store_key' column is set as the value of the 'ori_printer_store_key' column.

Otherwise, if the value of the column 'enrollment_type' is not equal to 'Kitless', then the value of the 'store_key' column is set as the value of the 'ori_card_store_key' column.

If none of the above conditions match, the value of the 'store_key' column is set as the value of the 'ori_printer_store_key' column.

4.Create bounty rate column which has 2 values kit bounty or kitless bounty. And it depends on compensation type column.

5.We add mapped store logic for MSH. Mapped store will be true only for these conditions (Updated - DE only, not AT):

retailer_kit_printer should be equal to MSH.

country_id should be DE (Note: AT is NOT included for iPaper).

When country_id = DE and store_comments_text starts with M OR S and store_comments_text is not null and store display name is not null.

For any other retailer or country, mapped_store is set to None.

26)Decimal Round Off Columns Created -

1.We create retailer mix round off column by rounding off retailer mix into 2 decimal places.

****ipaper_amount column calculation:****

We create ipaper_amount column based on these conditions:

- When country_id is US: ipaper_amount = ipaper_bounty (from retailer rules)
- When country_id is not US: ipaper_amount = retailer_mix_round_off * ipaper_bounty

The ipaper_bounty value comes from the retailer rules table (ipaper_bounty column).

27)USD Conversion -

1.We define dollar conversion table:

Table : team_iide_prod.gold_css_dim.dim_currency_rate (Updated from App_BM_Instant_Ink_Ops.base_hist_curr_val_v2)

Logic : We join dollar conversion table to convert the local currency amount to USD. The table is pivoted by currency_code to get conversion rates for each currency.

28)Gross for US -

1.We define the gross up table for bounty,

Table : team_iide_prod.gold_ii_pii.printer_consents_country_agg_iisb (Updated from bi_fact.printer_consents_country_agg_iisb)

We take the latest gross up value by partitioning according to country_id, program type,reporting_month,order by created_at.

Logic : We create gross up column from amount by dividing the amount with telemetry_optin_and_blank_percent and multiply with 100.

This is done because US comes under phase one logic. We gross up the amount with the percentage present in this table which is updated every month.

2.1st we convert all amount to USD then gross up only for US and convert the amount gross up column to USD.

****NEW - MSH Germany Grossup Logic:****

For MSH Germany (DE), a special consent percentage calculation is applied:

1. get_consent_percent_msh_bounty function calculates the consent percentage:
 - consent_yes_count: Sum of retailer_mix_round_off where retailer_kit_printer = 'MSH'
 - consent_total_count: Sum of retailer_mix_round_off where allocated_retailer = 'MSH'
 - consent_percent = consent_yes_count / consent_total_count
2. update_amount_grossup_bounty function applies the grossup:
 - For MSH DE subscribers: retailer_mix_round_off_grossup = retailer_mix_round_off / consent_percent
 - ipaper_amount_grossup for DE = retailer_mix_round_off_grossup * 2
3. Final conversion to USD is applied for the grossed-up amounts.

29)Check point -4

This end check point 4, all these data come into df4.

Final bounty given for enrollees.

30)End